Entity Set Co-Expansion in StackOverflow

Yu Zhang[1*], Yunyi Zhang[1*], Yucheng Jiang[1], Martin Michalski[1],
Yu Deng[2], Lucian Popa[3], ChengXiang Zhai[1], Jiawei Han[1]
[1]Department of Computer Science, University of Illinois at Urbana-Champaign, Urbana, IL, USA
[2]IBM Thomas J. Watson Research Center, Yorktown Heights, NY, USA
[3]IBM Almaden Research Center, San Jose, CA, USA
{yuz9, yzhan238, yj17, martinm6, czhai, hanj}@illinois.edu, {dengy, lpopa}@us.ibm.com

***Abstract*—Given a few seed entities of a certain type (e.g., SOFTWARE or PROGRAMMING LANGUAGE), entity set expansion aims to discover an extensive set of entities that share the same type as the seeds. Entity set expansion in software-related domains such as StackOverflow can benefit many downstream tasks (e.g., software knowledge graph construction) and facilitate better IT operations and service management. Meanwhile, existing approaches are less concerned with two problems: (1) How to deal with multiple types of seed entities simultaneously? (2) How to leverage the power of pre-trained language models (PLMs)? Being aware of these two problems, in this paper, we study the entity set co-expansion task in StackOverflow, which extracts LIBRARY, OS, APPLICATION, and LANGUAGE entities from StackOverflow question-answer threads. During the co-expansion process, we use PLMs to derive embeddings of candidate entities for calculating similarities between entities. Experimental results show that our proposed SECOEXPAN framework outperforms previous approaches significantly.**

***Index Terms*—set expansion, entity extraction, StackOverflow**

I. INTRODUCTION

The task of entity set expansion [4], [7] aims to enrich a small set of seed entities (e.g., "*Java*", "*C++*", and "*PHP*") by extracting other entities belonging to the same type (e.g., "*Python*", "*SQL*", and "*JavaScript*" that are also programming languages) from a large corpus. Previous studies have shown the benefit of entity set expansion to a wide range of downstream applications, such as named entity recognition [10], taxonomy construction [8], and text classification [15].

While existing approaches demonstrate their effectiveness in Wikipedia articles, news, and scientific papers, entity set expansion in software-related texts, such as StackOverflow question-answer threads and GitHub issue reports, has been largely unexplored. Yet, there is an increasing interest in extracting software-related entities, which is a fundamental step towards software knowledge graph construction and can benefit IT operations and service management. For example, in the StackOverflowNER dataset [9], 20 types of entities, including LIBRARY, OS, APPLICATION, and LANGUAGE, are annotated for entity-centric studies.

From the technical perspective, we identify two problems that are less concerned by existing studies: (1) *How to deal with multiple types of seed entities simultaneously?* To construct a heterogeneous knowledge graph, one needs to extract multiple types of entities. If more than one type of seeds are provided for set expansion, then for a given entity type, seeds from other types can serve as negative examples and help determine the expansion boundary. For example, given two sets of seeds {"*Java*", "*C++*", "*PHP*"} and {"*Windows*", "*iOS*", "*Ubuntu*"}, we know that the three OS seeds do not belong to the LANGUAGE type, and all the entities extracted for OS during expansion should be far from LANGUAGE as well. Without such guidance, the expansion process may suffer from semantic drifting and entity intrusion [2]. (2) *How to leverage the power of pre-trained language models?* Pre-trained language models (PLMs) such as BERT [1] have achieved significant performance improvement in a wide spectrum of text mining tasks by learning contextualized word embeddings. The generic knowledge learned by PLMs from web-scale corpora may complement the signals we can obtain from the input corpus. For example, for some less popular programming languages such as "*Kotlin*" and "*Groovy*", their occurrences may not be very frequent in the input corpus, in which case their semantics cannot be accurately learned solely from local contexts. In comparison, PLMs may have learned some knowledge of them from their Wikipedia pages during pre-training.

Contributions. In this paper, we aim to tackle the aforementioned two problems of entity set expansion and apply our framework to the StackOverflow domain. To be specific, first, we study the task of *entity set co-expansion*, which takes multiple types of seed entities as input and expands them simultaneously. We propose a framework, called SECOEXPAN, that iteratively expands each entity set while keeping mutual exclusivity of all types. Second, to utilize PLMs, we feed sentences containing the candidate entities into BERTOverflow [9] to derive entity representations based on both the entity themselves and their contexts. The obtained representations then play a key role in calculating similarities between entities during our co-expansion process. We conduct experiments on four entity types – LIBRARY, OS, APPLICATION, and LANGUAGE – from the StackOverflowNER dataset [9], and show that our proposed SECOEXPAN framework outperforms strong baselines including SetExpan [7] and CGExpan [14].

II. RELATED WORK

There have been many studies on entity set expansion. For a more detailed review of related work, please refer to [6]. Early studies such as EgoSet [4] and SetExpan [7] iteratively bootstrap the entity set by selecting skip-gram features and ranking new entities. Later, SetExpander [3] and CaSE [11] propose to capture distributional similarity between words

*Equal Contribution.

during expansion based on context-free word embeddings. More recently, CGExpan [14] enhances entity set expansion with language model probing. In contrast, our SECoExpan framework utilizes PLMs in a different way by obtaining contextualized embeddings of candidate entities.

III. Problem Definition

Our task is formally defined as follows.

Definition 1. (Entity Set Co-Expansion) *Given a corpus $\mathcal{D}$ and multiple entity sets $\mathcal{E}_1, \mathcal{E}_2, ..., \mathcal{E}_M$ describing M different entity types, where each entity set contains several (e.g., 5-10) seed entities belonging to one entity type (i.e., $\mathcal{E}_i = \{e_{i,1}, ..., e_{i,N}\}$), our task is to extract a set of new entities $\widetilde{\mathcal{E}}_i = \{e_{i,N+1}, ..., e_{i,N+K}\}$ from $\mathcal{D}$ for each entity type.*

To perform entity set co-expansion, one needs to first extract a candidate pool $\mathcal{P}$ of noun phrases from $\mathcal{D}$, which can be done by applying common phrase mining tools [5].

IV. The SECoExpan Framework

In this section, we first discuss how we utilize a large pre-trained language model to get entity embeddings. Then, we will introduce our new entity set co-expansion method.

A. Entity Embeddings with Masked Language Model

We first use a PLM to get entity embeddings based on the context information provided in the corpus. The masked language modeling (MLM) task is first proposed in BERT [1] as a training objective for large PLMs. Basically, it is a cloze-filling task such that the model is trained to recover some tokens that are randomly selected and replaced by the special `[MASK]` token in the input sentences. Such a model after pre-training shows superior language representation power by capturing contextualized information.

However, PLMs can only embed single tokens in its fixed vocabulary, while entities are often multi-token phrases (e.g., "*Windows XP*"). Therefore, we introduce two strategies to get entity embeddings with PLMs [12], [13]. (1) Given an entity e and a sentence s containing it, we can get its *content embedding* $\mathbf{h}_{e|s}^{\text{content}}$ by feeding the original sentence into a PLM. Since the entity may be tokenized into multiple tokens by the model (e.g., "*Windows*" and "*XP*"), we take the average of all its corresponding token's output embeddings as its context embedding for this sentence. (2) Then, for the same entity and sentence, we can also get its *context embedding* $\mathbf{h}_{e|s}^{\text{context}}$ by replacing the entire entity with the `[MASK]` token. Then, the output embedding of the `[MASK]` token is used as its context embedding, because PLMs can only see its surrounding context to infer its semantic.

Since the above strategies can only get entity embeddings based on one sentence, we further leverage the corpus to get corpus-level entity embeddings. For each candidate entity e, we find the complete set $\mathcal{S}_e$ of sentences containing it from the corpus. Then, we get its content and context embeddings for each sentence and then take the average over all sentences to get the corpus-level embeddings.

$$\mathbf{h}_e^{\text{content}} = \frac{1}{|\mathcal{S}_e|} \sum_{s \in \mathcal{S}_e} \mathbf{h}_{e|s}^{\text{content}},$$

$$\mathbf{h}_e^{\text{context}} = \frac{1}{|\mathcal{S}_e|} \sum_{s \in \mathcal{S}_e} \mathbf{h}_{e|s}^{\text{context}}.$$

We can also get a third type of embeddings by concatenating $\mathbf{h}_e^{\text{content}}$ and $\mathbf{h}_e^{\text{context}}$ to capture both signals.

$$\mathbf{h}_e^{\text{both}} = [\mathbf{h}_e^{\text{content}}; \mathbf{h}_e^{\text{context}}].$$

We will study the effects of using each type of embeddings for the set expansion task in the experiments.

B. Iterative Entity Set Co-Expansion

After getting the PLM-based embeddings for each entity, $\mathbf{h}_e^X \quad (X \in \{\text{content}, \text{context}, \text{both}\})$, we propose an entity set co-expansion method based on the embeddings.

We first define a similarity score between an entity set $\mathcal{E}$ and a candidate entity e as the average of cosine similarity between e and each entity currently in $\mathcal{E}$. That is

$$\text{sim}(e, \mathcal{E}) = \frac{1}{|\mathcal{E}|} \sum_{e' \in \mathcal{E}} \cos(\mathbf{h}_e^X, \mathbf{h}_{e'}^X).$$

Because we have multiple entity sets to expand simultaneously, for one target set, the remaining ones from different semantic classes can serve as its negative examples (i.e., irrelevant entities) to guide the expansion process. To be specific, after calculating the similarity score between each pair of candidate entity and entity set, we compare the scores for each candidate entity and select the set with the maximum score, which we name *the matched set of an entity*. For example, if a candidate entity has a similarity score of 0.7 with expanded Library entities and a score of 0.3 with all other entity sets, we will view the Library entity set as its matched set and OS, Application, and Language entities as irrelevant ones. Formally, given a candidate entity e and all entity sets to expand $\mathcal{E}_1, \mathcal{E}_2, ..., \mathcal{E}_M$, the matched set of e is

$$\mathcal{E}_e^* = \underset{\mathcal{E}_i, i \in \{1,...,M\}}{\arg\max} \ \text{sim}(e, \mathcal{E}_i).$$

By doing so, each entity will only be expanded to the set with the highest score. Note that for ambiguous entities that are relevant to more than one type, its highest and second highest scores may be close, in which case directly picking the highest one can be risky. To tackle this problem, we propose an iterative framework. In each iteration, only very top-ranked candidate entities will be expanded. Since it is difficult to simultaneously achieve very high similarities with more than one type, ambiguous entities are unlikely to be expanded under this strategy.

The complete set co-expansion method is as follows: given all the current entity sets and the candidate entities, we first calculate the similarity score for each pair of candidate entity and entity set and find the matched set of each entity as defined

TABLE I
SELECTED SEEDS FOR EACH TYPE.

Type	Seeds
LIBRARY	"*jquery*", "*api*", "*angular*", "*django*", "*spring*"
OS	"*windows*", "*android*", "*linux*", "*ios*", "*ubuntu*"
APPLICATION	"*browser*", "*mysql*", "*git*", "*chrome*", "*excel*", "*visual studio*"
LANGUAGE	"*javascript*", "*java*", "*php*", "*html*", "*c++*", "*sql*", "*python*"

above. Then, we will select top-k (e.g., $k = 10$) entities $\mathcal{T}^k \subseteq \mathcal{P}$ with the highest scores to their matched sets.

$$\mathcal{T}^k = \underset{\mathcal{T}\subseteq\mathcal{P},|\mathcal{T}|=k}{\arg\max} \sum_{e\in\mathcal{T}} \text{sim}(e, \mathcal{E}_e^*).$$

Each entity of $\mathcal{T}^k$ will then be used to expand its matched set. After expanding these k entities, we can re-calculate the entity-entity set scores with the updated sets and repeat the entity selection process, which becomes an iterative expansion framework. Finally, the expansion process will stop after all sets reach a given target size t.

V. EXPERIMENTS

A. Dataset

Our corpus $\mathcal{D}$ consists of two parts: one is the StackOverflowNER dataset [9] (but we do not use the annotations inside as supervision), and the other is a sampled subcorpus of the Stack Exchange data dump[1] which has ~1.26M questions and answers.

We consider four entity types – LIBRARY, OS, APPLICATION, and LANGUAGE. For each type, we select 5-7 seed entities which are the most frequently annotated ones in StackOverflowNER. The selected seeds are listed in Table I.

To get the candidate entity pool from the corpus, we apply a phrase mining tool, AutoPhrase [5], to first get all quality phrases. Then, we use spaCy[2] to only keep those noun phrases. Finally, we get 48,178 entities in the candidate pool.

B. Compared Methods

We compare the following entity set expansion methods.

- **SetExpan** [7]: This method iteratively expands the entity sets by selecting skip-gram context features and scoring entities with a rank ensemble method.
- **CGExpan** [14]: This method uses a pre-trained language model to predict the type name of each entity set and use the name to guide the expansion process.
- **SECOEXPAN-content**: This is a variant of our proposed method using *content* embeddings only.
- **SECOEXPAN-context**: This is a variant of our proposed method using *context* embeddings only.
- **SECOEXPAN-both**: This is a variant of our proposed method using concatenations of *both* embeddings.

[1]https://archive.org/download/stackexchange/stackoverflow.com-Posts.7z
[2]https://spacy.io

TABLE II
P@k SCORES OF ALL COMPARED METHODS.

Methods	P@10	P@20	P@30
SetExpan [7]	0.325	0.350	0.358
CGExpan [14]	0.775	0.725	0.708
SECOEXPAN			
-content	0.825	0.763	0.692
-context	0.825	**0.850**	**0.842**
-both	**0.925**	0.837	0.742

C. Evaluation Metric

We use Precision@K (P@K) as our evaluation metric. To be specific, for each entity type, we check how many of the top-K extracted new entities $\widetilde{\mathcal{E}}_i = \{e_{i,N+1}, ..., e_{i,N+K}\}$ belong to the same entity type as $\mathcal{E}_i$. Then, we compute the average proportions across all entity types. Formally, if we use $e \sim \mathcal{E}_i$ to denote that the entity e has the same type as the seeds in $\mathcal{E}_i$, then P@K can be defined as

$$\text{P@}K = \frac{1}{M}\sum_{i=1}^{M}\frac{1}{K}\sum_{j=1}^{K}\mathbf{1}(e_{i,N+j} \sim \mathcal{E}_i),$$

where $\mathbf{1}(\cdot)$ is the indicator function.

D. Hyperparameters and Implementation

We use BERTOverflow [9] as the PLM to get entity embeddings, which is a BERT-base model fine-tuned on the StackOverflowNER corpus. The target expansion size t is set to 30. In each iteration of SECOEXPAN, top-10 (i.e., $k = 10$) entities are selected to expand the entity sets before re-calculating the similarity scores in the next iteration.

E. Performance Comparison

Table II shows the P@K scores of all compared methods, where $K = 10$, 20, and 30. From Table II, we can observe that: (1) SECOEXPAN-context and SECOEXPAN-both consistently and significantly outperform the baselines, indicating the effectiveness of our proposed framework. Using skip-gram features only, SetExpan performs not so well. Enhanced by the power of PLMs, CGExpan achieves much better performance than SetExpan, but still underperforms SECOEXPAN in most cases. This is mainly because CGExpan does not have a specific design to expand multiple types of entities simultaneously. (2) Among the three variants of SECOEXPAN, SECOEXPAN-both has the highest P@10 score, while SECOEXPAN-context has the highest P@20 and P@30 scores. This observation implies that content information can only benefit the precision of top-ranked entities, while context information is more useful for extracting accurate lower-ranked entities. We will explain the reason for this through a case study.

F. Case Study

Table III shows the expanded entity sets of SetExpan, SECOEXPAN-both, SECOEXPAN-content, and SECOEXPAN-context for two types, where we list both top-ranked entities (i.e., 1st to 5th) and lower-ranked ones (i.e., 26th to 30th). We can see that: (1) SetExpan picks many general terms (e.g., "*a*", "*local*", "*modern*") that do not belong to any specific entity

TABLE III
EXPANDED ENTITY SETS FOR OS AND LANGUAGE TYPES, WITH ERRONEOUS ENTITIES COLORED RED .

Entity Type	Seed Entity Set	SetExpan		SECoExpan-both		SECoExpan-content		SECoExpan-context	
OS	{"*windows*", "*android*", "*linux*", "*ios*", "*ubuntu*"}	1	"*window*"	1	"*ms windows*"	1	"*microsoft window*"	1	"*macos*"
		2	"*iphone*"	2	"*microsoft window*"	2	"*gnu linux*"	2	"*osx*"
		3	"*a*"	3	"*gnu linux*"	3	"*window server*"	3	"*macosx*"
		4	"*mac*"	4	"*window ce*"	4	"*windows server*"	4	"*mac*"
		5	"*local*"	5	"*arch linux*"	5	"*ms windows*"	5	"*mac osx*"
			...		...		...		...
		26	"*modern*"	26	"*window server 2008*"	26	"*window 2008 server*"	26	"*debian jessie*"
		27	"*office*"	27	"*window powershell*"	27	"*windows api*"	27	"*windows vista*"
		28	"*regular*"	28	"*window phone*"	28	"*window explorer*"	28	"*nix*"
		29	"*4 gb*"	29	"*windows 8*"	29	"*window mobile*"	29	"*windows 7*"
		30	"*remote*"	30	"*windows powershell*"	30	"*windows 8*"	30	"*window vista*"
LANGUAGE	{"*javascript*", "*java*", "*php*", "*html*", "*c++*", "*sql*", "*python*"}	1	"*c*"	1	"*c*"	1	"*objective c*"	1	"*c*"
		2	"*js*"	2	"*objective c*"	2	"*obj c*"	2	"*vb*"
		3	"*css*"	3	"*vb net*"	3	"*c*"	3	"*go*"
		4	"*ruby*"	4	"*obj c*"	4	"*c sharp*"	4	"*golang*"
		5	"*korean*"	5	"*c sharp*"	5	"*turbo c*"	5	"*elixir*"
			...		...		...		...
		26	"*a*"	26	"*asp net webapi*"	26	"*java net urlconnection*"	26	"*sml*"
		27	"*haskell*"	27	"*dot net*"	27	"*js main js*"	27	"*racket*"
		28	"*clojure*"	28	"*asp net core webapi*"	28	"*discord js*"	28	"*vhdl*"
		29	"*powershell*"	29	"*dot net core*"	29	"*tcp ip*"	29	"*nim*"
		30	"*vb net*"	30	"*asp net boilerplate*"	30	"*crypto js*"	30	"*scala*"

type. (2) For SECoExpan-both and SECoExpan-content, because we use entity content (i.e., tokens in each entity) to derive embeddings, the extracted terms are more likely to have lexical overlap with the seed entities. For top-ranked entities (e.g., "*ms windows*", "*gnu linux*", "*c*"), this strategy yields high precision. However, when it comes to lower-ranked entities (e.g., "*windows powershell*", "*windows api*"), lexical overlap does not necessarily indicate that two entities belong to the same type. This explains our observation from Table II that content information is not as robust as context information for extracting lower-ranked entities.

VI. CONCLUSIONS AND FUTURE WORK

In this paper, we study the problem of entity set co-expansion in StackOverflow to extract LIBRARY, OS, APPLICATION, and LANGUAGE entities with just a few seeds. We propose to leverage a PLM to derive entity embeddings based on entity content and/or context. Then, an iterative co-expansion framework is proposed to simultaneously enrich multiple sets of entities based on the calculated entity embeddings. Experimental results show that our proposed SECoExpan framework significantly outperforms strong baselines such as SetExpan and CGExpan. Through quantitative and qualitative analyses, we also conclude that context signals are more robust than content signals for extracting lower-ranked entities. For future work, first, it is of our interest to generalize our framework to more types of entities such as VERSION and DEVICE. Second, we would like to explore the possibility of applying our entity set co-expansion results to distantly supervised or few-shot named entity recognition in StackOverflow and GitHub issue reports.

ACKNOWLEDGMENTS

We thank anonymous reviewers for their valuable and insightful feedback. This work was supported by the IBM-Illinois Discovery Accelerator Institute and National Science Foundation IIS-19-56151, IIS-17-41317, and IIS 17-04532.

REFERENCES

[1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT'19*, pages 4171–4186, 2019.
[2] J. Huang, Y. Xie, Y. Meng, J. Shen, Y. Zhang, and J. Han. Guiding corpus-based set expansion by auxiliary sets generation and co-expansion. In *WWW'20*, pages 2188–2198, 2020.
[3] J. Mamou, O. Pereg, M. Wasserblat, A. Eirew, Y. Green, S. Guskin, P. Izsak, and D. Korat. Term set expansion based nlp architect by intel ai lab. In *EMNLP'18: system demonstrations*, pages 19–24, 2018.
[4] X. Rong, Z. Chen, Q. Mei, and E. Adar. Egoset: Exploiting word ego-networks and user-generated ontology for multifaceted set expansion. In *WSDM'16*, pages 645–654, 2016.
[5] J. Shang, J. Liu, M. Jiang, X. Ren, C. R. Voss, and J. Han. Automated phrase mining from massive text corpora. *IEEE TKDE*, 30(10):1825–1837, 2018.
[6] J. Shen and J. Han. *Automated Taxonomy Discovery and Exploration*. Springer Nature, 2022.
[7] J. Shen, Z. Wu, D. Lei, J. Shang, X. Ren, and J. Han. Setexpan: Corpus-based set expansion via context feature selection and rank ensemble. In *ECML-PKDD'17*, pages 288–304, 2017.
[8] J. Shen, Z. Wu, D. Lei, C. Zhang, X. Ren, M. T. Vanni, B. M. Sadler, and J. Han. Hiexpan: Task-guided taxonomy construction by hierarchical tree expansion. In *KDD'18*, pages 2180–2189, 2018.
[9] J. Tabassum, M. Maddela, W. Xu, and A. Ritter. Code and named entity recognition in stackoverflow. In *ACL'20*, pages 4913–4926, 2020.
[10] X. Wang, Y. Zhang, Q. Li, X. Ren, J. Shang, and J. Han. Distantly supervised biomedical named entity recognition with dictionary expansion. In *BIBM'19*, pages 496–503, 2019.
[11] P. Yu, Z. Huang, R. Rahimi, and J. Allan. Corpus-based set expansion with lexical features and distributed representations. In *SIGIR'19*, pages 1153–1156, 2019.
[12] Y. Zhang, F. Guo, J. Shen, and J. Han. Unsupervised key event detection from massive text corpora. In *KDD'22*, pages 2535–2544, 2022.
[13] Y. Zhang, Y. Meng, X. Wang, S. Wang, and J. Han. Seed-guided topic discovery with out-of-vocabulary seeds. In *NAACL'22*, pages 279–290, 2022.
[14] Y. Zhang, J. Shen, J. Shang, and J. Han. Empower entity set expansion via language model probing. In *ACL'20*, pages 8151–8160, 2020.
[15] Y. Zhang, F. F. Xu, S. Li, Y. Meng, X. Wang, Q. Li, and J. Han. Higitclass: Keyword-driven hierarchical classification of github repositories. In *ICDM'19*, pages 876–885, 2019.